

Apostila 1 — Fundamentos de Python para Economia (Teoria Completa)

Capítulo 1 — Por que Python

1.1. Um pouco de história

Python foi criado por Guido van Rossum e lançado em 1991. O nome não vem da cobra, mas do grupo de comédia britânico *Monty Python's Flying Circus* — van Rossum queria uma linguagem “divertida de usar”. Essa origem desprestigiada explica um traço central da linguagem: ela foi desenhada para ser lida por humanos primeiro, e executada por máquinas depois.

Isso está formalizado no **Zen of Python**, um conjunto de 19 princípios que qualquer instalação de Python exhibe ao rodar `import this`. Dois deles resumem o espírito da linguagem:

“Readability counts.” (Legibilidade importa) *“There should be one — and preferably only one — obvious way to do it.”* (Deve haver uma — e de preferência só uma — forma óbvia de fazer algo)

Para um economista, isso importa mais do que parece: um script de análise de dados frequentemente é lido meses depois — por você mesmo, por um coautor, por um parecerista — e precisa ser compreensível sem arqueologia de código.

1.2. Por que economistas migraram para Python

Até os anos 2010, o ferramental padrão de um economista era Excel para planilhas, Stata ou EViews para econometria, e talvez MATLAB para modelos computacionais. Cada uma dessas ferramentas resolve bem um problema específico, mas nenhuma resolve todos:

Ferramenta	Ponto forte	Limitação
Excel	Visual, fácil de começar	Não escala (arquivos grandes travam), difícil de auditar, erros silenciosos em fórmulas
Stata/EViews	Comandos econométricos prontos	Licença paga, menos flexível para dados não tabulares, comunidade menor de desenvolvimento
MATLAB	Forte em álgebra linear/otimização	Licença cara, pouco usado fora de engenharia/finanças quantitativas
R	Excelente para estatística	Sintaxe menos amigável para programação geral, ecossistema mais fragmentado
Python	Faz tudo isso e mais (dados, gráficos, econometria, machine learning, web, automação)	Curva de aprendizado inicial um pouco maior que Excel

Três fatores concretos explicam a adoção maciça de Python em bancos centrais (BCB, Fed, BCE), organismos internacionais (FMI, Banco Mundial, OCDE) e departamentos de economia:

1. **Gratuidade e abertura.** Python é open source. Um pesquisador em qualquer lugar do mundo pode rodar exatamente o mesmo código sem pagar licença — o que também torna a pesquisa **reprodutível**: outro economista pode reexecutar sua análise linha por linha.
2. **Ecossistema unificado.** As mesmas bibliotecas (pandas, numpy, statsmodels, scikit-learn) cobrem desde a limpeza de uma planilha até a estimação de um VAR ou o treino de um modelo preditivo. Você não precisa trocar de ferramenta a cada etapa do trabalho.
3. **Automação.** Um script Python pode ser reexecutado automaticamente — todo mês, ao sair um novo IPCA, por exemplo — sem intervenção manual. Isso é o que torna possível um dashboard macroeconômico que se atualiza sozinho.

1.3. O que você vai construir com isso

Ao final desta série de apostilas, o objetivo é que você consiga: baixar dados econômicos reais (BCB, IBGE, FMI), limpá-los e organizá-los, visualizá-los de forma profissional, rodar regressões e modelos de séries temporais, e empacotar tudo isso em projetos que podem virar um TCC, um relatório de research ou um portfólio.

Tudo começa, porém, na base: entender como o Python pensa. Esse é o assunto desta apostila.

Capítulo 2 — Como o Python executa seu código

2.1. Linguagem interpretada vs. compilada

Linguagens como C ou Rust são **compiladas**: existe uma etapa prévia em que todo o código-fonte é traduzido de uma vez para instruções de máquina, gerando um executável. Só depois disso o programa roda.

Python é **interpretado**: um programa chamado interpretador lê seu código linha por linha (mais precisamente, instrução por instrução) e vai executando cada uma imediatamente, sem gerar um executável separado. Isso tem duas consequências práticas:

- **Feedback mais rápido.** Você escreve uma linha, roda, vê o resultado — sem esperar uma compilação. É por isso que Python é tão usado para exploração de dados: o ciclo “tentei algo → vi o resultado → ajustei” é curtíssimo.
- **Erros aparecem só quando a linha é executada.** Um script pode ter um erro de sintaxe na linha 200 e ainda assim rodar perfeitamente até chegar lá, porque o interpretador não analisa o arquivo inteiro de antemão da mesma forma que um compilador analisaria.

Tecnicamente, o interpretador CPython (a implementação padrão de Python) primeiro traduz seu código para uma representação intermediária chamada **bytecode**, e depois executa esse bytecode numa máquina virtual. Isso não muda o comportamento do ponto de vista prático, mas explica por que arquivos `.pyc` aparecem em pastas `__pycache__` — são o bytecode cacheado.

2.2. Três formas de rodar Python

Ambiente	Quando usar
Script (arquivo.py)	Análises que você quer repetir, versionar e automatizar
REPL (terminal interativo, python)	Testar uma linha rapidamente, sem salvar nada
Notebook (Jupyter)	Exploração de dados, onde você quer ver gráficos e tabelas ao lado do código, célula por célula

Um arquivo `.py` é executado de cima para baixo, instrução por instrução:

```
# isso é um comentário – o computador ignora
# serve para explicar o código, não para ser executado
```

```
print("Hello, Economia!") # print exhibe algo na tela
```

Salve como teste.py e execute pelo terminal:

```
python teste.py
```

Saída:

```
Hello, Economia!
```

2.3. print() — sua ferramenta de diagnóstico mais usada

print() exhibe qualquer valor na tela. Antes de existirem debuggers sofisticados, é a forma mais rápida (e ainda a mais usada no dia a dia) de entender o que um programa está fazendo em cada etapa.

```
print("IPCA de junho: 0.16%")
```

```
# print aceita múltiplos valores, separados por vírgula
mes = "junho"
valor = 0.16
print("IPCA de", mes, ":", valor, "%")
```

Um hábito recomendado: sempre que um resultado não for o esperado, adicione print() antes e depois da linha suspeita para ver os valores intermediários. Isso é chamado, informalmente, de “debugging por print”, e resolve a maioria dos problemas de um iniciante.

Capítulo 3 — Variáveis e o modelo de memória

3.1. O que realmente é uma variável

Em muitas linguagens (C, por exemplo), uma variável é uma **caixa** com um espaço fixo de memória, e atribuir um valor é colocar algo dentro dessa caixa. Em Python, o modelo é diferente e vale a pena entender desde já, porque evita confusões mais adiante (especialmente com listas, no Capítulo 7).

Em Python, uma variável é um **nome (rótulo)** que aponta para um objeto na memória. Atribuir um valor não coloca nada “dentro” da variável — cria um objeto em algum lugar da memória e faz o nome apontar para ele.

```
ipca_junho = 0.16
selic = 10.50
pib_2025 = 3.2
```

```
print(ipca_junho)    # 0.16
print(selic)         # 10.5
```

Aqui, ipca_junho não é uma caixa contendo 0.16 — é um rótulo colado no objeto 0.16, que existe na memória do computador. Essa distinção parece sutil agora, mas explica comportamentos que confundem iniciantes, como o fato de duas variáveis poderem apontar para o **mesmo** objeto (voltaremos a isso com listas).

3.2. Regras e convenções de nomenclatura

Regras (o Python exige, senão dá erro): - Pode conter letras, números e underscore _ - Não pode começar com número - Não pode ser uma palavra reservada (if, for, class, etc.) - Maiúsculas e minúsculas são diferentes (IPCA ≠ ipca)

Convenções (o Python não exige, mas a comunidade segue — o guia oficial de estilo chama-se **PEP 8**): - Nomes de variáveis em `snake_case`: `taxa_juros`, não `taxaJuros` nem `TaxaJuros` - Nomes claros em vez de abreviações obscuras: `taxa_juros` é melhor que `tj` - Constantes (valores que não mudam) em maiúsculas: `TAXA_SELIC_META = 10.50`

Seguir PEP 8 não é purismo estético: em economia, código é frequentemente compartilhado (com coautores, orientadores, revisores), e nomes claros substituem metade da documentação que você precisaria escrever.

3.3. Tipagem dinâmica

Python é **dinamicamente tipado**: uma variável não tem um tipo fixo — o tipo pertence ao objeto ao qual ela aponta no momento, e pode mudar:

```
x = 10          # x aponta para um int
print(type(x)) # <class 'int'>

x = "dez"      # agora x aponta para um str
print(type(x)) # <class 'str'>
```

Isso contrasta com linguagens **estaticamente tipadas** (C, Java), onde uma variável declarada como inteiro só pode guardar inteiros para sempre. A tipagem dinâmica de Python dá flexibilidade e reduz a verbosidade do código, mas exige disciplina: como o interpretador não vai barrar você antecipadamente, erros de tipo (somar texto com número, por exemplo) só aparecem quando a linha problemática é executada — não antes.

3.4. Exemplo Resolvido — Rastreamento de rótulos

Enunciado: sem rodar no computador, preveja o que o código abaixo imprime e explique o porquê usando o modelo de “rótulo” (não “caixa”) apresentado neste capítulo.

```
selic = 10.5
selic_copia = selic
selic = 11.0
print(selic_copia)
```

Solução comentada:

A saída é 10.5.

1. `selic = 10.5` cria o objeto 10.5 na memória e faz o rótulo `selic` apontar para ele.
2. `selic_copia = selic` **não copia o número** — cria um segundo rótulo, `selic_copia`, apontando para o **mesmo objeto** 10.5 que `selic` já aponta. Neste momento, os dois rótulos apontam para o mesmo lugar.
3. `selic = 11.0` cria um objeto **novo**, 11.0, e faz o rótulo `selic` passar a apontar para ele. O objeto antigo (10.5) continua existindo na memória, e `selic_copia` continua apontando para ele — porque reatribuir `selic` só move o rótulo `selic`, não afeta nenhum outro rótulo.
4. Por isso `print(selic_copia)` ainda mostra 10.5.

Isso funciona de forma “segura” porque números (`int`, `float`) são **imutáveis**: reatribuir sempre cria um objeto novo, nunca modifica o objeto existente. No Capítulo 7 (Listas), veremos que esse raciocínio muda quando o objeto é **mutável** — lá, alterar através de um rótulo *afeta* o outro rótulo que aponta para o mesmo objeto.

Capítulo 4 — Tipos de dados primitivos

4.1. Números inteiros (int)

```
ano = 2026
populacao = 214_000_000 # underscore é só separador visual, ignorado pelo Python
```

Diferentemente de linguagens como C, o int de Python não tem limite de tamanho fixo (32 ou 64 bits) — ele cresce conforme necessário, limitado apenas pela memória do computador. Isso significa que, em Python, você pode multiplicar números arbitrariamente grandes sem *overflow*.

4.2. Números decimais (float) e um alerta importante

```
ipca = 0.16
cambio = 5.45
pib_per_capita = 45000.00
```

Ponto flutuante não é exato. Computadores representam números decimais em base binária, e frações decimais simples (como 0,1) não têm representação binária finita — da mesma forma que 1/3 não tem representação decimal finita. O resultado é um pequeno erro de arredondamento:

```
print(0.1 + 0.2)          # 0.30000000000000004 — não é 0.3!
print(0.1 + 0.2 == 0.3)   # False
```

Isso **não é um bug do Python** — é uma característica de como todo computador representa ponto flutuante (IEEE 754), presente em praticamente qualquer linguagem.

Por que isso acontece, exatamente. Um computador guarda um float em base **binária** (potências de 2), não em base decimal (potências de 10). Em base 10, uma fração como 1/3 não tem representação finita (0,333...); em base 2, o mesmo problema ocorre com frações que são triviais em base 10, como 0,1. Expandindo 0,1 como soma de potências de 2:

$$0,1 = \frac{1}{16} + \frac{1}{32} + \frac{1}{256} + \dots = \sum_{i=1}^{\infty} a_i \cdot 2^{-i}$$

essa série **nunca fecha exatamente** — é uma dízima binária periódica, análoga a $\frac{1}{3} = 0,333\dots$ em base 10. O computador guarda apenas um número finito de bits (52 bits de mantissa em IEEE 754 double precision), então armazena a **melhor aproximação possível** de 0,1, não o valor exato. Ao somar duas aproximações (0,1 e 0,2), os erros de arredondamento — cada um da ordem de 10^{-17} — se acumulam e aparecem no resultado. Você pode ver a representação binária real com:

```
from decimal import Decimal
print(Decimal(0.1)) # 0.1000000000000000055511151231257827021181583404541015625
```

Ou seja: 0.1 do Python **já nasce** ligeiramente diferente de $\frac{1}{10}$ exato — a soma só torna esse erro, que já existia antes, visível.

Para um economista, a implicação prática é: nunca compare valores monetários ou taxas com == diretamente; arredonde antes, ou compare a diferença absoluta com uma tolerância pequena:

```
tolerancia = 1e-9
print(abs((0.1 + 0.2) - 0.3) < tolerancia) # True
```

4.3. Texto (str)

```
pais = "Brasil"
sigla = 'BR' # aspas simples ou duplas — tanto faz, desde que combinem no início e fim
mensagem = "0 PIB cresceu 3.2% em 2025"
```

Strings em Python são sequências de caracteres Unicode, o que significa suporte nativo a acentos e caracteres especiais (ã, ç, €) sem configuração extra — relevante ao trabalhar com nomes de países, cidades ou variáveis em português.

4.4. Booleano (bool)

```
em_recessao = False
crescendo = True
```

Um detalhe técnico útil: em Python, bool é tecnicamente uma subclasse de int, onde True equivale a 1 e False equivale a 0. Isso permite somar booleanos diretamente — útil, por exemplo, para contar quantos meses uma condição foi verdadeira:

```
meses_em_alta = [True, True, False, True]
print(sum(meses_em_alta)) # 3 — soma True como 1 e False como 0
```

4.5. Verificando e convertendo tipos

```
print(type(ipca))          # <class 'float'>
print(type(pais))          # <class 'str'>
print(type(em_recessao))  # <class 'bool'>

# Conversão explícita (casting)
print(int("2026"))         # 2026 (str -> int)
print(float("5.45"))       # 5.45 (str -> float)
print(str(0.16))           # "0.16" (float -> str)
print(int(5.9))            # 5 (float -> int trunca, não arredonda!)
```

Atenção: int() sobre um float **trunca** (descarta a parte decimal), não arredonda. int(5.9) dá 5, não 6. Para arredondar de verdade, use round(5.9).

4.6. Exemplo Resolvido — Convertendo valor monetário em formato brasileiro

Enunciado: valores monetários brasileiros costumam vir formatados como "R\$ 1.234,56" (ponto para milhar, vírgula para decimal) — o oposto do que float() entende. Escreva uma função valor_brl_para_float que converta essa string para um float utilizável em cálculos.

```
texto = "R$ 1.234,56"
```

Solução comentada:

```
def valor_brl_para_float(texto):
    limpo = texto.replace("R$", "").strip() # remove o prefixo e espaços: "1.234,56"
    limpo = limpo.replace(".", "")          # remove separador de milhar: "1234,56"
    limpo = limpo.replace(",", ".")         # troca vírgula decimal por ponto:
    "1234.56"
    return float(limpo)                    # agora float() entende: 1234.56

print(valor_brl_para_float("R$ 1.234,56")) # 1234.56
print(valor_brl_para_float("R$ 45,00"))    # 45.0
```

O raciocínio é sempre o mesmo ao “limpar” um dado antes de convertê-lo: primeiro remover o que **não é número** (símbolo de moeda, espaços), depois ajustar os separadores para o formato que Python reconhece (ponto decimal), e só então chamar float(). Essa sequência — limpar, ajustar, converter — reaparece o tempo todo ao importar planilhas de fontes brasileiras na Apostila 2.

Capítulo 5 — Operadores

5.1. Aritméticos

```
soma = 5 + 3      # 8
subtracao = 5 - 3  # 2
multiplicacao = 5 * 3 # 15
divisao = 5 / 3    # 1.666... (sempre retorna float)
divisao_inteira = 5 // 3 # 1 (descarta o resto)
resto = 5 % 3      # 2 (resto da divisão – o operador módulo)
potencia = 5 ** 3   # 125
```

```
# Aplicação econômica: juros compostos
valor_futuro = 1000 * (1 + 0.10) ** 5 # 1610.51
print(f"R$\ {valor_futuro:.2f}")
```

O operador módulo (%) parece pouco útil à primeira vista, mas aparece com frequência em análise de séries temporais — por exemplo, para descobrir se um mês é o final de trimestre (`mes % 3 == 0`).

A fórmula de juros compostos usada acima será formalizada com rigor na Apostila 6 (Matemática Financeira):

$$VF = VP \cdot (1 + i)^n$$

onde VF é o valor futuro, VP o valor presente, i a taxa de juros por período e n o número de períodos.

5.2. Comparação (resultado é sempre bool)

```
10 > 5    # True
10 < 5     # False
10 == 10   # True  (igualdade – repare que são dois sinais de igual)
10 != 5    # True  (diferente)
10 >= 10   # True
5 <= 3     # False
```

Um erro comum de iniciante é usar `=` (atribuição) quando o objetivo era `==` (comparação). `x = 10` atribui 10 a `x`; `x == 10` pergunta se `x` vale 10.

Python também permite **encadear comparações**, algo que a maioria das linguagens não faz:

```
selic = 10.5
print(0 < selic < 15) # True – equivalente a (0 < selic) and (selic < 15)
```

5.3. Lógicos e curto-circuito

```
True and False # False (os dois precisam ser True)
True or False  # True  (pelo menos um True)
not True       # False (inverte)
```

and/or em Python fazem **avaliação de curto-circuito**: em `a and b`, se `a` já for False, Python nem avalia `b` (o resultado já está decidido). Isso é usado, por exemplo, para evitar erros:

```
lista_vazia = []

# Sem curto-circuito, acessar lista_vazia[0] daria erro (IndexError)
if len(lista_vazia) > 0 and lista_vazia[0] > 10:
    print("primeiro elemento é grande")
# como len(lista_vazia) > 0 é False, Python nunca tenta avaliar lista_vazia[0]
```

Aplicação econômica:

```
inflacao = 0.60
desemprego = 8.5
```

```
if inflacao > 0.5 and desemprego > 8.0:
    print("Estagflação?")
```

5.4. Exemplo Resolvido — Juro real exato (Fisher) vs. aproximado

Enunciado: no Capítulo 5.1 usamos $\text{selic} - \text{ipca}$ como aproximação do juro real (equação de Fisher **aproximada**). A fórmula **exata** é:

$$i_{\text{real}} = \frac{1 + i_{\text{nominal}}}{1 + \text{inflação}} - 1$$

Escreva uma função `juro_real_exato(selic, ipca)` (ambos em % ao ano) e compare o resultado com a aproximação simples para $\text{selic} = 15.0$ e $\text{ipca} = 10.0$. A diferença é grande ou pequena?

Solução comentada:

```
def juro_real_exato(selic, ipca):
    i_nominal = selic / 100
    inflacao = ipca / 100
    i_real = (1 + i_nominal) / (1 + inflacao) - 1
    return i_real * 100

def juro_real_aproximado(selic, ipca):
    return selic - ipca

selic, ipca = 15.0, 10.0
print(f"Exato: {juro_real_exato(selic, ipca):.3f}%") # 4.545%
print(f"Aproximado: {juro_real_aproximado(selic, ipca):.3f}%") # 5.000%
```

A diferença ($\approx 0,45$ ponto percentual) parece pequena, mas em taxas altas ela cresce — é por isso que a fórmula aproximada ($\text{selic} - \text{ipca}$) só deve ser usada como estimativa rápida de cabeça, nunca em cálculo financeiro de precisão. O operador de potência ($**$) não aparece aqui, mas a estrutura da fórmula — divisão encadeada com parênteses — é o mesmo tipo de expressão aritmética do Capítulo 5.1, só que aplicada com rigor.

Capítulo 6 — Strings em detalhe

6.1. Strings são sequências imutáveis

Uma string é uma **sequência ordenada de caracteres** — e, como toda sequência em Python, pode ser indexada e fatiada com a mesma sintaxe que veremos em listas no próximo capítulo:

```
pais = "Brasil"
print(pais[0]) # "B" (primeiro caractere, índice 0)
print(pais[-1]) # "l" (último caractere)
print(pais[0:3]) # "Bra" (fatiamento)
```

O ponto crucial: strings são **imutáveis**. Uma vez criada, você não pode alterar um caractere dentro dela:

```
pais[0] = "b" # TypeError! strings não suportam atribuição de item
```

Qualquer operação que “parece” modificar uma string na verdade **cria uma string nova**:

```
pais_minusculo = pais.lower() # cria uma string nova; "pais" continua "Brasil"
```

6.2. Concatenação e f-strings

```
# Concatenação com +
pais = "Brasil"
ano = 2026
frase = pais + " em " + str(ano) # "Brasil em 2026"
# str() converte número para texto – obrigatório, já que + entre str e int dá erro

# F-string (a forma recomendada e mais legível)
frase2 = f"{pais} em {ano}"
print(frase2) # "Brasil em 2026"
```

F-strings (introduzidas no Python 3.6) permitem embutir expressões inteiras dentro de {}, não só variáveis:

```
selic = 10.5
ipca = 4.5
print(f"Juro real aproximado: {selic - ipca:.1f}%") # calcula e formata na mesma
linha
```

6.3. Mini-linguagem de formatação

O formato dentro de {valor:especificação} segue uma mini-linguagem própria de Python:

Especificação	Efeito	Exemplo
:.2f	2 casas decimais	f"{0.1667:.2f}" → "0.17"
:.0f	Sem casas decimais	f"{0.1667:.0f}" → "0"
:,	Separador de milhar	f"{1000000:,}" → "1,000,000"
:>10	Alinha à direita em 10 espaços	útil para alinhar colunas de números
:%	Formata como percentual	f"{0.045:.1%}" → "4.5%"

```
ipca = 0.1667
print(f"IPCA: {ipca:.2f}%") # "IPCA: 0.17%"
print(f"IPCA: {ipca:.1f}%") # "IPCA: 0.2%"

taxa = 0.045
print(f"Taxa: {taxa:.1%}") # "Taxa: 4.5%" – multiplica por 100 e adiciona %
```

6.4. Métodos úteis de string

```
texto = " Brasil "
texto.upper() # " BRASIL "
texto.lower() # " brasil "
texto.strip() # "Brasil" (remove espaços das pontas – essencial ao limpar
dados importados)
texto.replace("a", "o") # " Brosil "
texto.split("a") # [" Br", "sil "]
",".join(["a", "b", "c"]) # "a,b,c" – o inverso de split
```

strip() e split() merecem destaque especial: ao importar dados econômicos de um CSV ou de uma API, é comum vir texto com espaços indesejados (" Brasil ") ou valores concatenados ("2024-Q1") que precisam ser separados. Esses dois métodos resolvem 90% desses casos.

6.6. Exemplo Resolvido — Relatório formatado com alinhamento

Enunciado: dado o dicionário pib abaixo, imprima uma tabela de texto simples com o nome do país alinhado à esquerda em 12 caracteres e o valor alinhado à direita em 8 caracteres, com 1 casa decimal e símbolo de percentual.

```
pib = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1}
```

Solução comentada:

```
pib = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1}
```

```
for pais, valor in pib.items():
    print(f"{pais:<12}{valor:>8.1f}%")
```

Saída:

```
Brasil          2.2%
Argentina       -1.8%
Chile           3.1%
```

{pais:<12} alinha a string à **esquerda** (<) ocupando 12 caracteres — preenchendo com espaços à direita. {valor:>8.1f} alinha o número à **direita** (>) em 8 caracteres, com 1 casa decimal (.1f). Essa combinação — alinhamento + casas decimais — é exatamente o que gera tabelas de texto legíveis em relatórios impressos no terminal, sem precisar de nenhuma biblioteca externa.

Capítulo 7 — Listas

7.1. O que é uma lista

Uma lista é uma coleção **ordenada** e **mutável** de valores.

```
inflacoes = [0.50, 0.70, 0.88, 0.67, 0.58, 0.16]
paises = ["Brasil", "Argentina", "Chile", "Colômbia"]
mista = [1, "texto", True, 3.14] # tipos diferentes são permitidos, mas evite —
# dificulta manutenção
```

7.2. Indexação e fatiamento

```
# Acessando elementos (o índice começa em 0!)
inflacoes[0]    # 0.50 (primeiro)
inflacoes[1]    # 0.70 (segundo)
inflacoes[-1]   # 0.16 (último)
inflacoes[-2]   # 0.58 (penúltimo)

# Fatiamento (slicing) — [inicio:fim:passo]
inflacoes[0:3]  # [0.50, 0.70, 0.88] (índices 0, 1, 2 — o "fim" nunca é incluído)
inflacoes[:3]   # [0.50, 0.70, 0.88] (mesma coisa; início vazio = "desde o
# começo")
inflacoes[3:]   # [0.67, 0.58, 0.16] (do índice 3 até o fim)
inflacoes[::2]  # [0.50, 0.88, 0.58] (pulando de 2 em 2)
inflacoes[::-1] # [0.16, 0.58, 0.67, 0.88, 0.70, 0.50] (inverte a ordem)
```

A regra de que o índice final **nunca é incluído** é uma fonte comum de erro em iniciantes (chamado *off-by-one error*). Uma forma de memorizar: lista[a:b] tem exatamente b - a elementos.

7.3. Mutabilidade e o problema da aliasing

Diferente das strings, listas **são mutáveis**: você pode alterar um elemento no lugar, sem criar uma lista nova. Isso, combinado com o modelo de “variável como rótulo” do Capítulo 3, gera um comportamento que surpreende muitos iniciantes:

```
inflacoes_2024 = [0.5, 0.7, 0.9]
inflacoes_copia = inflacoes_2024 # isso NÃO cria uma cópia! copia só o rótulo

inflacoes_copia.append(1.2)
print(inflacoes_2024) # [0.5, 0.7, 0.9, 1.2] – mudou também!
```

`inflacoes_copia = inflacoes_2024` não copia os dados: cria um segundo rótulo apontando para o **mesmo** objeto lista na memória. Alterar um afeta o outro, porque só existe uma lista. Isso é chamado de **aliasing**. Para criar uma cópia de verdade:

```
inflacoes_copia = inflacoes_2024.copy() # ou list(inflacoes_2024), ou
inflacoes_2024[:]
inflacoes_copia.append(1.2)
print(inflacoes_2024) # [0.5, 0.7, 0.9] – agora não muda
print(inflacoes_copia) # [0.5, 0.7, 0.9, 1.2]
```

(Isso é uma cópia **rasa** — “shallow copy”. Se a lista contiver outras listas dentro, os elementos internos ainda seriam compartilhados. Para esse caso existe `copy.deepcopy()`, mas isso é raro no dia a dia de análise de dados.)

7.4. Métodos de lista

```
numeros = [1, 2, 3]
numeros.append(4) # [1, 2, 3, 4] (adiciona no fim)
numeros.insert(0, 0) # [0, 1, 2, 3, 4] (insere na posição 0)
numeros.remove(2) # [0, 1, 3, 4] (remove a primeira ocorrência do VALOR 2)
numeros.pop() # remove e retorna o último elemento
numeros.pop(0) # remove e retorna o elemento do índice 0
len(numeros) # tamanho da lista
sum(inflacoes) # soma
max(inflacoes) # maior valor
min(inflacoes) # menor valor
sorted(inflacoes) # nova lista ordenada (crescente) – não altera a original
sorted(inflacoes, reverse=True) # decrescente
```

Repare a diferença entre `remove()` (remove por **valor**) e `pop()` (remove por **índice**, e ainda devolve o valor removido) — os nomes parecidos costumam confundir.

7.6. Exemplo Resolvido — Corrigindo outliers sem afetar a lista original

Enunciado: dada uma lista de inflações mensais, escreva uma função `capar_outliers` que devolva uma **lista nova** onde qualquer valor acima de um limite seja substituído pelo próprio limite — sem alterar a lista original (evitando o problema de aliasing visto na Seção 7.3).

```
inflacoes = [0.50, 0.70, 3.50, 0.67, 0.58, 0.16] # 3.50 parece um erro de digitação
```

Solução comentada:

```
def capar_outliers(dados, limite):
    return [valor if valor <= limite else limite for valor in dados]

inflacoes = [0.50, 0.70, 3.50, 0.67, 0.58, 0.16]
inflacoes_corrigidas = capar_outliers(inflacoes, limite=1.0)

print(inflacoes) # [0.5, 0.7, 3.5, 0.67, 0.58, 0.16] – inalterada
print(inflacoes_corrigidas) # [0.5, 0.7, 1.0, 0.67, 0.58, 0.16]
```

O ponto central: a list comprehension **sempre constrói uma lista nova**, então não há risco de aliasing aqui — diferente de `inflacoes.append(...)` ou `inflacoes[2] = ...`, que alterariam o objeto original no lugar. Sempre que a intenção for “criar uma versão modificada, preservando os

dados originais”, prefira uma expressão que gere uma lista nova a um método que modifique a lista existente.

Capítulo 8 — Dicionários

8.1. O que é um dicionário e por que ele é rápido

Um dicionário armazena pares **chave** → **valor**. Diferente de uma lista, onde você acessa por posição numérica, num dicionário você acessa por uma chave que você mesmo escolhe (um nome, uma sigla, uma data).

```
pib = {
    "Brasil": 2.2,
    "Argentina": -1.8,
    "Chile": 3.1,
    "Colômbia": 2.8
}
```

Internamente, um dicionário Python é implementado como uma **tabela hash**: cada chave passa por uma função matemática (a *hash function*) que calcula diretamente onde o valor está guardado na memória. Isso significa que buscar `pib["Brasil"]` é praticamente instantâneo, independentemente de o dicionário ter 10 ou 10 milhões de entradas — bem diferente de procurar um valor numa lista, que no pior caso exige percorrer item por item.

Como isso funciona, em mais detalhe. A função `hash(hash("Brasil"))` em Python) transforma a chave num número inteiro grande; esse número, reduzido ao tamanho da tabela interna (via resto da divisão), aponta diretamente para o “balde” (*bucket*) onde o par chave-valor deveria estar. Isso é o que dá complexidade $O(1)$ — tempo constante, não crescente com o tamanho dos dados — para buscar, inserir ou remover, em contraste com a lista, cuja busca por valor é $O(n)$ (no pior caso, percorre todos os n elementos). Duas consequências práticas dessa implementação:

- **Colisões existem e são tratadas automaticamente.** Duas chaves diferentes podem, por acaso, mapear para o mesmo balde (`hash("Brasil") % tamanho_tabela == hash("Chile") % tamanho_tabela`); o Python resolve isso internamente, mas é o motivo de a complexidade $O(1)$ ser uma *média*, não uma garantia absoluta.
- **A chave precisa ser hasheável, e portanto imutável.** É por isso que listas não podem ser chave de dicionário (`TypeError: unhashable type: 'list'`) — se o conteúdo da chave pudesse mudar depois de guardada, seu hash mudaria, e o Python não conseguiria mais encontrá-la no balde certo. Tuplas (Capítulo 11.6), strings e números podem ser chave; listas e dicionários, não.

Na prática: **use dicionário sempre que a busca for por um identificador nomeado** (país, código, data), e lista quando a ordem sequencial for o que importa.

8.2. Acessando e modificando

```
print(pib["Brasil"])          # 2.2
print(pib.get("Brasil"))      # 2.2 — forma segura, não quebra se a chave não existir
print(pib.get("Uruguai", 0.0)) # 0.0 — valor padrão quando a chave não existe

# pib["Uruguai"] sem o .get() geraria um KeyError

# Adicionar/modificar
pib["Peru"] = 3.5             # adiciona uma chave nova
pib["Brasil"] = 2.5           # atualiza uma chave existente
```

8.3. Métodos e iteração

```
pib.keys()      # todas as chaves
pib.values()    # todos os valores
pib.items()     # pares (chave, valor)
```

```
for pais, valor in pib.items():
    print(f"{pais}: {valor}%")
```

8.4. Dicionários aninhados

É comum, em dados econômicos, ter mais de uma dimensão — por exemplo, PIB por país e por ano. A solução natural é um dicionário de dicionários:

```
pib_por_ano = {
    "Brasil": {"2023": 2.9, "2024": 3.1, "2025": 2.2},
    "Chile":  {"2023": 0.2, "2024": 2.6, "2025": 3.1},
}
```

```
print(pib_por_ano["Brasil"]["2024"]) # 3.1
```

```
for pais, series in pib_por_ano.items():
    for ano, valor in series.items():
        print(f"{pais} em {ano}: {valor}%")
```

Vale adiantar: na Apostila 2 (pandas), esse tipo de estrutura aninhada normalmente vira um DataFrame, que é mais adequado para manipular tabelas de verdade — mas entender o dicionário aninhado ajuda a entender o que o pandas faz por baixo dos panos.

8.5. Lista vs. dicionário — quando usar qual

Situação	Estrutura recomendada
Sequência de valores na ordem em que ocorreram (ex: IPCA mês a mês)	Lista
Valores identificados por nome (ex: PIB por país)	Dicionário
Preciso acessar por posição (“o terceiro valor”)	Lista
Preciso acessar por identificador (“o valor do Chile”)	Dicionário

8.6. Exemplo Resolvido — Combinando dois dicionários

Enunciado: dados dois dicionários com o PIB do mesmo conjunto de países em anos diferentes, escreva uma função `variacao_percentual` que retorne um **novo dicionário** {país: variação} com a diferença entre os dois anos, país por país.

```
pib_2024 = {"Brasil": 3.1, "Chile": 2.6, "Peru": 3.3}
pib_2025 = {"Brasil": 2.2, "Chile": 3.1, "Peru": 2.9}
```

Solução comentada:

```
def variacao_percentual(dados_ano1, dados_ano2):
    resultado = {}
    for pais in dados_ano1:
        resultado[pais] = dados_ano2[pais] - dados_ano1[pais]
```

```

    return resultado

pib_2024 = {"Brasil": 3.1, "Chile": 2.6, "Peru": 3.3}
pib_2025 = {"Brasil": 2.2, "Chile": 3.1, "Peru": 2.9}

print(variacao_percentual(pib_2024, pib_2025))
# {'Brasil': -0.9, 'Chile': 0.5, 'Peru': -0.4}

```

Repare que `for pais in dados_ano1` itera diretamente sobre as **chaves** do dicionário (isso é equivalente a `for pais in dados_ano1.keys()`, mas mais direto). A função pressupõe que os dois dicionários têm exatamente as mesmas chaves; se isso não for garantido, o acesso `dados_ano2[pais]` poderia lançar um `KeyError` — o Capítulo 13 mostra como blindar esse tipo de acesso.

Capítulo 9 — Condicionais

9.1. if / elif / else

```

ipca = 0.88

if ipca > 1.0:
    print("Inflação alta")
elif ipca > 0.5:
    print("Inflação moderada")
elif ipca > 0.2:
    print("Inflação controlada")
else:
    print("Deflação")

```

O Python avalia as condições **em ordem**, de cima para baixo, e executa apenas o **primeiro** bloco cuja condição seja verdadeira — mesmo que uma condição posterior também fosse verdadeira. Por isso a ordem de um `elif` importa.

9.2. Truthiness — o que conta como “verdadeiro”

Em condicionais, Python aceita qualquer valor, não apenas `True/False` — e converte implicitamente para booleano. Essa conversão segue uma regra: valores “vazios” ou “zero” são considerados falsos (*falsy*); praticamente tudo o mais é considerado verdadeiro (*truthy*).

```

lista_vazia = []
lista_com_dados = [0.5]

if lista_vazia:
    print("tem dados")
else:
    print("lista vazia")    # imprime isso — lista vazia é falsy

if 0:
    print("nunca executa")    # 0 é falsy
if 0.0:
    print("nunca executa")    # 0.0 é falsy
if "":
    print("nunca executa")    # string vazia é falsy
if "0":
    print("executa!")         # string "0" (não vazia) é truthy!

```

O último exemplo é uma armadilha clássica: a string `"0"` é *truthy*, mesmo representando “zero”, porque o que importa é ela **não estar vazia**.

9.3. Operador ternário

```
status = "Alta" if ipca > 0.5 else "Baixa"
```

Isso é equivalente a um if/else de quatro linhas, mas condensado numa expressão — útil quando o valor precisa ser atribuído diretamente a uma variável.

9.4. Combinando condições

```
pib = -0.5
desemprego = 12.0

if pib < 0 and desemprego > 10:
    print("País em recessão com alto desemprego")
elif pib < 0 or desemprego > 10:
    print("Um dos indicadores está negativo")
```

9.5. Exemplo Resolvido — Tratando dado ausente antes de comparar

Enunciado: é comum que uma série econômica tenha meses sem dado disponível, representados como None (o “vazio” do Python). Escreva uma função `classifica_com_dado_ausente(ipca)` que retorne "sem dado" se ipca for None, e classifique normalmente ("alta" se > 0.5 , senão "controlada") caso contrário — **sem** deixar o programa quebrar ao comparar None com um número.

```
valores = [0.6, None, 0.3, 0.9]
```

Solução comentada:

```
def classifica_com_dado_ausente(ipca):
    if ipca is None:          # sempre confira None ANTES de comparar com números
        return "sem dado"
    elif ipca > 0.5:
        return "alta"
    else:
        return "controlada"

valores = [0.6, None, 0.3, 0.9]
for v in valores:
    print(classifica_com_dado_ausente(v))
# alta / sem dado / controlada / alta
```

O detalhe crítico é a **ordem**: `ipca is None` precisa vir antes de `ipca > 0.5`, porque `None > 0.5` gera um `TypeError` em Python (não existe comparação de ordem entre None e um número — diferente de outras linguagens onde isso silenciosamente viraria `False`). Usar `is None` (em vez de `== None`) é a forma idiomática recomendada, porque None é um valor único e especial na linguagem, e `is` verifica identidade exata, não apenas igualdade.

Capítulo 10 — Laços de repetição

10.1. for — percorrendo sequências (iteráveis)

Listas, strings, dicionários e vários outros tipos em Python compartilham a propriedade de serem **iteráveis**: dá para percorrê-los item por item com `for`. Isso não é coincidência — é um protocolo geral da linguagem, e por isso a sintaxe do `for` é a mesma independentemente do tipo de dado percorrido:

```
# Percorrer lista
inflacoes = [0.50, 0.70, 0.88, 0.67]
for valor in inflacoes:
```

```

    print(f"IPCA: {valor}%")

# Percorrer com índice, usando enumerate()
for i, valor in enumerate(inflacoes):
    print(f"Mês {i+1}: {valor}%")

# Percorrer dicionário
pib = {"Brasil": 2.2, "Chile": 3.1}
for pais, valor in pib.items():
    print(f"{pais}: {valor}%")

# range() gera uma sequência de números – útil para repetir N vezes
for i in range(5):
    print(i) # 0, 1, 2, 3, 4 (começa em 0, para antes de 5)

for i in range(1, 13):
    print(f"Mês {i}") # 1 a 12

```

10.2. while — repetir enquanto uma condição for verdadeira

```

saldo = 1000
taxa = 0.01
anos = 0

while saldo < 2000:
    saldo *= (1 + taxa)
    anos += 1

print(f"Leva {anos} anos para dobrar")

# CUIDADO: loop infinito! Sempre garanta que a condição
# vai se tornar False em algum momento – aqui, saldo cresce a cada
# iteração e eventualmente ultrapassa 2000.

```

Regra prática: use for quando souber, de antemão, sobre o que está iterando (uma lista, um intervalo fixo). Use while quando a repetição depende de uma condição que só se resolve durante a execução (como “até o saldo dobrar” — não se sabe quantas iterações de antemão).

10.3. break e continue

```

# break – interrompe o loop imediatamente
for valor in inflacoes:
    if valor > 0.8:
        print("Alta detectada, parando")
        break
    print(valor)

# continue – pula para a próxima iteração, sem executar o resto do bloco
for valor in inflacoes:
    if valor < 0.3:
        continue # pula valores baixos
    print(valor)

```

10.4. Nota sobre desempenho: laços vs. vetorização

Um for em Python percorre elementos um a um, o que é intuitivo mas relativamente lento para volumes grandes de dados (milhões de linhas). Bibliotecas como numpy e pandas (Apostila 2) permitem **vetorizar** a mesma operação — aplicá-la a um array inteiro de uma vez, usando código otimizado em C por baixo dos panos. Por enquanto, for é a ferramenta certa para aprender os

conceitos; na prática profissional com bases de dados grandes, você vai preferir a versão vetorizada sempre que ela existir.

10.5. Exemplo Resolvido — Trajetória de patrimônio mês a mês

Enunciado: um investidor aplica R\$ 500 por mês (aporte constante) a uma taxa de 0,8% ao mês. Construa, com um `for`, a lista com o saldo acumulado ao final de cada um dos 12 meses (aporte entra **antes** de render no mês).

Solução comentada:

```
def trajetoria_patrimonio(aporte_mensal, taxa, meses):
    saldo = 0.0
    trajetoria = []
    for _ in range(meses):
        saldo += aporte_mensal          # aporte do mês entra primeiro
        saldo *= (1 + taxa)            # depois rende sobre o saldo já com o aporte
        trajetoria.append(round(saldo, 2))
    return trajetoria

historico = trajetoria_patrimonio(aporte_mensal=500, taxa=0.008, meses=12)
for mes, saldo in enumerate(historico, start=1):
    print(f"Mês {mes:2d}: R$ {saldo:.2f}")
```

O uso de `_` como nome de variável no `for _ in range(meses)` é uma convenção do Python para dizer “este valor existe, mas eu não preciso dele” — aqui, só precisamos repetir o bloco `meses` vezes, sem usar o índice diretamente. Note também `enumerate(historico, start=1)`: o parâmetro `start` desloca a contagem para começar em 1 em vez de 0, evitando o +1 manual usado em exemplos anteriores.

Capítulo 11 — Funções

11.1. Por que funções importam além de “reaproveitar código”

Uma função é um bloco de código reutilizável — mas o motivo mais importante para usá-las, em um contexto de análise econômica, não é apenas evitar repetição: é **isolar e nomear uma etapa do raciocínio**. Uma função bem nomeada, como `taxa_juros_real(selic, ipca)`, documenta a si mesma; um bloco solto de operações não.

Funções também são a base da **reprodutibilidade**: se o cálculo de uma variável está dentro de uma função testável, você pode verificar que ela produz o resultado certo isoladamente, sem rodar todo o script.

```
def nome_da_funcao(parametro1, parametro2):
    """Docstring: explica o que a função faz"""
    resultado = parametro1 + parametro2
    return resultado
```

11.2. Função sem parâmetros e sem retorno

```
def saudacao():
    print("Bem-vindo ao analisador econômico")

saudacao()
```

11.3. Parâmetros e retorno

```
def calcular_cambio_real(pib, cambio):
    """Calcula taxa de câmbio real simplificada"""
```

```

        return cambio * (1 + pib / 100)

cambio_real = calcular_cambio_real(3.2, 5.45)
print(f"Câmbio real ajustado: {cambio_real:.2f}")

```

11.4. Vários parâmetros

```

def taxa_juros_real(selic, ipca_esperado):
    """Calcula taxa de juros real (Fischer simplificado)"""
    return selic - ipca_esperado

juro_real = taxa_juros_real(10.5, 4.5)
print(f"Juro real: {juro_real:.1f}%")

```

11.5. Parâmetros com valor padrão — e uma armadilha a evitar

```

def pib_per_capita(pib_total, populacao, unidade="mil"):
    """Calcula PIB per capita com unidade configurável"""
    resultado = pib_total / populacao
    if unidade == "mil":
        return resultado / 1000
    elif unidade == "milhao":
        return resultado / 1_000_000
    return resultado

print(pib_per_capita(2_000_000_000_000, 214_000_000))
print(pib_per_capita(2_000_000_000_000, 214_000_000, unidade="milhao"))

```

Valores padrão são convenientes, mas há uma armadilha conhecida: **nunca use uma lista ou dicionário como valor padrão**. Por causa do modelo de memória visto no Capítulo 3, o objeto padrão é criado **uma única vez**, na definição da função — e não a cada chamada:

```

# ERRADO — bug clássico
def adiciona_pais(pais, lista=[]):
    lista.append(pais)
    return lista

print(adiciona_pais("Brasil"))      # ["Brasil"]
print(adiciona_pais("Chile"))       # ["Brasil", "Chile"] — a lista "padrão" foi
reaproveitada!

# CORRETO
def adiciona_pais_corrigido(pais, lista=None):
    if lista is None:
        lista = []
    lista.append(pais)
    return lista

```

11.6. Retornando múltiplos valores

```

def estatisticas_basicas(dados):
    """Retorna média, mínimo e máximo de uma lista"""
    media = sum(dados) / len(dados)
    minimo = min(dados)
    maximo = max(dados)
    return media, minimo, maximo # na verdade, retorna uma tupla (media, minimo,
maximo)

inf = [0.5, 0.7, 0.88, 0.67, 0.58, 0.16]

```

```
media, min_inf, max_inf = estatisticas_basicas(inf)
print(f"Média: {media:.2f}, Mín: {min_inf:.2f}, Máx: {max_inf:.2f}")
```

11.7. Escopo: variáveis locais vs. globais

Uma variável criada **dentro** de uma função só existe dentro dela (escopo local); ela desaparece quando a função termina e não é visível fora:

```
def calcula_dobro(x):
    resultado = x * 2 # "resultado" é local a esta função
    return resultado

calcula_dobro(5)
print(resultado) # NameError! "resultado" não existe fora da função
```

Isso é desejável: cada função funciona como uma “caixa fechada”, e nomes internos não colidem com nomes usados em outras partes do script — o que torna o comportamento de uma função **previsível** independentemente de quem a chama.

11.8. Exemplo Resolvido — Função pura vs. função com efeito colateral

Enunciado: compare as duas funções abaixo, que aparentemente fazem “a mesma coisa” (adicionar um registro de PIB a uma lista de histórico). Explique por que a primeira é mais segura de usar em um script de análise, e qual problema a segunda pode causar.

```
historico = []
```

```
def registra_impura(pais, valor):
    historico.append((pais, valor)) # modifica uma lista de FORA da função
    return historico

def registra_pura(historico_atual, pais, valor):
    return historico_atual + [(pais, valor)] # retorna uma lista NOVA
```

Solução comentada:

`registra_impura` tem um **efeito colateral**: ela modifica `historico`, uma variável definida fora da função, sem receber isso explicitamente como parâmetro. Isso funciona, mas torna o comportamento da função dependente de um estado externo escondido — se você chamar `registra_impura` em partes diferentes do script, o resultado depende de quantas vezes ela já foi chamada antes, o que dificulta testar a função isoladamente.

`registra_pura` é uma **função pura**: seu resultado depende *apenas* dos parâmetros recebidos, e ela não modifica nada fora de si mesma (`historico_atual + [...]` cria uma lista nova, não altera a original — o mesmo princípio de “criar em vez de mutar” do Exemplo Resolvido do Capítulo 7). Isso a torna previsível e fácil de testar: dado o mesmo input, sempre produz o mesmo output.

```
historico = []
historico = registra_pura(historico, "Brasil", 2.2)
historico = registra_pura(historico, "Chile", 3.1)
print(historico) # [('Brasil', 2.2), ('Chile', 3.1)]
```

Na prática profissional, funções puras são preferidas sempre que possível — especialmente em pipelines de dados, onde rastrear “quem alterou o quê” em funções impuras é uma fonte comum de bugs difíceis de reproduzir.

Capítulo 12 — List comprehension

12.1. Sintaxe e motivação

List comprehension é uma forma concisa de criar listas a partir de outra sequência, condensando em uma linha o que um for levaria três ou quatro linhas para fazer. É amplamente usada em código Python idiomático, inclusive dentro do próprio pandas.

```
# Sintaxe: [expressao for item in lista if condicao]

inf = [0.5, 0.7, 0.88, 0.67]

# Aplicar uma transformação a cada elemento
dobro = [x * 2 for x in inf]          # [1.0, 1.4, 1.76, 1.34]

# Filtrar elementos
altos = [x for x in inf if x > 0.6]  # [0.7, 0.88, 0.67]

# Aplicar uma função a cada elemento
def taxar(x):
    return x * 1.1

taxado = [taxar(x) for x in inf]
```

O equivalente, sem list comprehension, ao exemplo de filtro:

```
altos = []
for x in inf:
    if x > 0.6:
        altos.append(x)
```

12.2. Quando (não) usar

List comprehension é ideal para transformações simples de uma linha. Quando a lógica interna fica complexa (múltiplas condições, vários passos), um for tradicional costuma ser mais legível — lembre do princípio “legibilidade importa” do Capítulo 1. Regra prática: se a comprehension não cabe confortavelmente numa linha, prefira o for explícito.

12.3. Exemplo Resolvido — Achatando um dicionário aninhado

Enunciado: reaproveite o `pib_por_ano` do Capítulo 8.4 (dicionário de dicionários). Use list comprehension **aninhada** para transformá-lo em uma única lista de tuplas (`país`, `ano`, `valor`) — uma operação comum ao preparar dados aninhados para virarem uma tabela (o que a Apostila 2 fará automaticamente com pandas).

```
pib_por_ano = {
    "Brasil": {"2023": 2.9, "2024": 3.1},
    "Chile": {"2023": 0.2, "2024": 2.6},
}
```

Solução comentada:

```
tabela = [
    (país, ano, valor)
    for país, series in pib_por_ano.items()
    for ano, valor in series.items()
]

print(tabela)
```

```
# [('Brasil', '2023', 2.9), ('Brasil', '2024', 3.1),  
#  ('Chile', '2023', 0.2), ('Chile', '2024', 2.6)]
```

Uma comprehension pode ter **mais de um for**, executados na mesma ordem em que apareceriam se fossem loops aninhados escritos à mão:

```
tabela = []  
for pais, series in pib_por_ano.items():  
    for ano, valor in series.items():  
        tabela.append((pais, ano, valor))
```

As duas versões produzem exatamente o mesmo resultado. A versão em comprehension é mais compacta; a versão com for explícito costuma ser mais fácil de depurar quando a lógica cresce (por exemplo, se fosse necessário um if adicional dentro do loop interno).

Capítulo 13 — Erros e depuração

13.1. Lendo uma mensagem de erro

Quando algo dá errado, Python interrompe a execução e mostra um **traceback**: o caminho que o programa percorreu até o erro, terminando na causa. A regra mais importante: **leia a última linha primeiro** — ela contém o tipo do erro e a mensagem; as linhas acima mostram em qual arquivo e linha do seu código o problema ocorreu.

```
# NameError – variável não definida  
print(valor_inexistente)  
# NameError: name 'valor_inexistente' is not defined  
  
# TypeError – operação com tipo errado  
"IPCA: " + 0.16  
# TypeError: can only concatenate str (not "float") to str  
"IPCA: " + str(0.16) # correto  
  
# IndexError – índice fora do intervalo da lista  
lista = [1, 2]  
lista[5]  
# IndexError: list index out of range  
  
# KeyError – chave que não existe no dicionário  
d = {"a": 1}  
d["b"]  
# KeyError: 'b'  
  
# ZeroDivisionError – divisão por zero  
10 / 0  
# ZeroDivisionError: division by zero
```

13.2. Tratando erros com try/except

Às vezes um erro é **esperado** (por exemplo, um dado que pode ou não existir) e você não quer que ele pare todo o script. Para isso existe try/except:

```
def taxa_segura(dados, chave):  
    try:  
        return dados[chave]  
    except KeyError:  
        print(f"Aviso: '{chave}' não encontrado, usando 0.0")  
        return 0.0
```

```
pib = {"Brasil": 2.2}
print(taxa_segura(pib, "Brasil"))    # 2.2
print(taxa_segura(pib, "Uruguai"))  # Aviso: ... / 0.0
```

try/except não deve virar hábito para “esconder” erros de programação (isso mascara bugs reais) — o uso correto é para situações genuinamente esperadas, como dados ausentes numa fonte externa. Sobre esse assunto — validação e tratamento de dados reais — voltaremos com mais profundidade na Apostila 2.

13.3. Exemplo Resolvido — Tratando dois tipos de erro diferentes

Enunciado: escreva uma função `carrega_taxa(dados, chave)` que busca uma taxa num dicionário e a converte para float. Ela deve lidar com **dois** problemas possíveis: a chave pode não existir (KeyError), ou o valor pode ser um texto não numérico como "N/D" (ValueError ao converter). Em ambos os casos, retorne None — mas imprima uma mensagem diferente para cada causa.

```
dados = {"Brasil": "4.5", "Chile": "N/D"}
```

Solução comentada:

```
def carrega_taxa(dados, chave):
    try:
        return float(dados[chave])
    except KeyError:
        print(f"Aviso: '{chave}' não encontrado na base")
        return None
    except ValueError:
        print(f"Aviso: valor de '{chave}' não é numérico ({dados[chave]!r})")
        return None

dados = {"Brasil": "4.5", "Chile": "N/D"}
print(carrega_taxa(dados, "Brasil"))    # 4.5
print(carrega_taxa(dados, "Chile"))     # Aviso: valor não é numérico / None
print(carrega_taxa(dados, "Peru"))      # Aviso: não encontrado / None
```

Um try pode ter **múltiplos except**, cada um capturando um tipo de erro diferente — Python testa cada except na ordem em que aparece e executa o primeiro que corresponder ao erro ocorrido. Isso permite dar um diagnóstico específico para cada causa possível de falha, em vez de um except genérico que trataria “chave ausente” e “valor inválido” da mesma forma, escondendo qual dos dois problemas realmente aconteceu.

Capítulo 14 — Boas práticas de código

Um script curto de 10 linhas não exige disciplina; um projeto que você vai reabrir em três meses, sim. Alguns hábitos, baratos de adotar desde já, evitam retrabalho:

- **Siga PEP 8:** snake_case para variáveis e funções, 4 espaços de indentação (nunca tab), linhas de até ~80-100 caracteres.
- **Nomeie por significado, não por tipo:** inflacao_mensal, não lista1 ou x2.
- **DRY (Don't Repeat Yourself):** se você copiou e colou um bloco de código três vezes mudando só um número, isso deveria ser uma função com um parâmetro.
- **Comente o “porquê”, não o “o quê”:** # ajuste porque o BCB muda a metodologia do IPCA em 2020 é útil; # soma 1 a x (quando o código já diz isso) não é.
- **Funções pequenas e com um único propósito:** se o nome da função tem “e” no meio (calcula_e_salva_e_plota), considere dividir em três funções.

Capítulo 15 — Exercícios para Executar (na mão)

Estes exercícios não exigem computador: o objetivo é treinar a leitura e o raciocínio sobre o código antes de escrevê-lo. Resolva no papel. **As soluções não estão neste documento** — quando terminar, peça para eu conferir suas respostas.

Exercício 1 — Trace de execução

Sem rodar no computador, escreva o que cada print abaixo exibiria, na ordem:

```
x = 10
y = 3
print(x // y)
print(x % y)
print(x / y)
x, y = y, x
print(x, y)
```

Exercício 2 — Juros compostos na mão

Calcule, sem usar código, o valor futuro de um capital de R\$ 500,00 aplicado a uma taxa de 2% ao mês, por 3 meses, usando a fórmula $VF = VP \cdot (1 + i)^n$. Mostre o cálculo mês a mês.

Exercício 3 — Avaliação lógica

Classifique cada expressão abaixo como True ou False, sem rodar no computador:

- a) `5 > 3 and 2 > 4`
- b) `5 > 3 or 2 > 4`
- c) `not (5 > 3)`
- d) `0 and (10 / 0)` # cuidado com curto-circuito!
- e) `"" or "Brasil"`
- f) `[] and [1, 2]`

Exercício 4 — Encontre o erro

Cada trecho abaixo tem um erro. Sem rodar, diga qual é o tipo de erro (NameError, TypeError, IndexError, KeyError ou ZeroDivisionError) e por quê.

- a) `preco = "10" + 5`
- b) `paises = ["Brasil", "Chile"]`
`print(paises[2])`
- c) `pib = {"Brasil": 2.2}`
`print(pib["Argentina"])`
- d) `print(taxa_cambio)`
- e) `resultado = 100 / (5 - 5)`

Exercício 5 — Pseudocódigo de decisão

Descreva, em português estruturado (pseudocódigo, sem precisar ser Python válido), a lógica para classificar o regime de política monetária: se a Selic estiver acima da meta de inflação mais 5 pontos percentuais, é “contracionista”; se estiver entre a meta e a meta + 5, é “neutra”; caso contrário, é “expansionista”. Depois, traduza seu pseudocódigo para Python.

Capítulo 16 — Exercícios para Executar (em código)

Implemente e execute cada um no seu editor. **As soluções não estão neste documento** — o objetivo é você rodar de verdade e ver o resultado; quando terminar, peça para eu revisar seu código.

Exercício 1 — Inflação acumulada

Crie uma função `inflacao_acumulada` que recebe uma lista de valores mensais (em %) e calcula a inflação acumulada no período, compondo cada taxa (não somando).

Exercício 2 — Maior e menor PIB

Dado um dicionário com PIB de países, retorne o país com maior PIB e o com menor.

```
pibs = {"Brasil": 2.2, "Argentina": -1.8, "Chile": 3.1, "Peru": 2.5, "México": 1.9}
```

Exercício 3 — Simulação de juros compostos mês a mês

Simule juros compostos: comece com R\$ 1000, taxa de 1% ao mês, por 12 meses. Mostre o saldo a cada mês.

Exercício 4 — Valores acima da média

Crie uma função que recebe uma lista de números e retorna uma nova lista apenas com valores acima da média.

Exercício 5 — Média geométrica do IPCA

A inflação acumulada de N meses equivale a compor as taxas mensais; a taxa mensal **média** equivalente não é a média aritmética, é a média geométrica. Crie uma função `ipca_medio_geometrico(taxas)` que recebe uma lista de taxas mensais (%) e retorna a taxa mensal média que, composta N vezes, reproduz a mesma inflação acumulada.

Exercício 6 — Meses em recessão

Dada uma lista com a variação percentual do PIB trimestre a trimestre, conte quantos trimestres consecutivos, no pior trecho, o país ficou em recessão técnica (2 trimestres seguidos de PIB negativo já configura recessão técnica — mas aqui queremos o maior número de trimestres negativos consecutivos, mesmo que ultrapasse 2).

```
pib_trimestral = [0.5, -0.2, -0.3, -0.1, 0.8, -0.5, 1.0]
```

Exercício 7 — Crescimento a partir de dicionário aninhado

Usando o dicionário aninhado abaixo, crie uma função que recebe o dicionário, um país e dois anos, e retorna a variação percentual do PIB entre esses dois anos (não a variação ano a ano — a mudança acumulada entre os dois pontos).

```
pib_por_ano = {
    "Brasil": {"2022": 3.0, "2023": 2.9, "2024": 3.1, "2025": 2.2},
    "Chile": {"2022": 2.4, "2023": 0.2, "2024": 2.6, "2025": 3.1},
}
```

Exercício 8 — Entrada validada com try/except

Crie uma função `taxa_para_float` que recebe uma string representando uma taxa (ex: "4.5") e retorna o valor como float. Se a conversão falhar (ex: texto não numérico, como "N/D"), a função deve capturar o erro e retornar None em vez de quebrar o programa.

Exercício 9 — Classificador de cenário macroeconômico

Combine condicionais e funções: crie `classifica_cenario(pib, inflacao, desemprego)` que retorna uma string descrevendo o cenário, usando as regras: PIB negativo e inflação alta (>5%) → "estagflação"; PIB positivo e inflação baixa (<3%) e desemprego baixo (<8%) → "expansão saudável"; qualquer outro caso → "cenário misto".

Exercício 10 — Séries “lado a lado” com list comprehension

Dadas duas listas de mesma extensão (IPCA e SELIC mês a mês), use list comprehension para criar uma terceira lista com o juro real aproximado (SELIC – IPCA) de cada mês, e depois outra lista comprehension para filtrar apenas os meses em que o juro real foi negativo.

```
selic = [10.5, 10.5, 10.75, 10.75, 11.0]
ipca = [4.2, 4.5, 4.8, 5.1, 11.5]
```

Exercício 11 — Limpando dados com taxas inválidas

Reaproveite a função `taxa_para_float` do Exercício 8. Dada a lista abaixo, crie uma função `limpar_taxas` que devolva apenas os valores que puderam ser convertidos (descartando os inválidos), sem quebrar o programa.

```
taxas_brutas = ["4.5", "3.2", "N/D", "5.1", "", "2.8"]
```

Exercício 12 — Fechamentos de trimestre

Dada uma lista com 24 meses (índice 0 = janeiro do ano 1), use `enumerate` e o operador módulo para retornar apenas os **números dos meses** (1 a 24) que representam fechamento de trimestre (múltiplos de 3).

Exercício 13 — Parseando um rótulo de período

Strings como "2024-Q1: 3.2%" aparecem com frequência em bases de dados exportadas de forma “suja”. Crie uma função `parse_periodo` que recebe essa string e retorna uma tupla (ano, trimestre, valor), com ano e trimestre como str e valor como float.

Exercício 14 — Semestres via fatiamento

Dada uma série de 12 valores mensais de IPCA, use *slicing* (sem loop) para retornar duas listas: os 6 primeiros meses e os 6 últimos.

Exercício 15 — Cópia segura de uma base

Escreva uma função `atualiza_pib_seguro(base, pais, novo_valor)` que recebe um dicionário {pais: pib}, retorna uma **cópia nova** com o valor de pais atualizado, e garante que o dicionário original passado como argumento não seja alterado (evite o aliasing visto no Capítulo 7/8).

Exercício 16 — Contador de cenários

Reaproveite `classifica_cenario` do Exercício 9. Dada uma lista de tuplas (pib, inflacao, desemprego), uma por trimestre, use um dicionário como **contador** para descobrir quantas vezes cada cenário ocorreu ao longo da série.

Exercício 17 — Estatísticas sem bibliotecas

Implemente `media_e_desvio(dados)` que retorna a média e o desvio padrão populacional de uma lista, **sem usar statistics nem numpy** — apenas o que foi visto neste capítulo (loops, list comprehension, sum, len, ** 0.5).

$$\sigma = \sqrt{\frac{\sum (x_i - \bar{x})^2}{n}}$$

Exercício 18 — Meses até dobrar (busca com while)

Generalize o exemplo do Capítulo 10: escreva `meses_para_dobrar(taxa_mensal)` que recebe uma taxa em decimal (ex: 0.01 para 1%) e retorna quantos meses são necessários para um capital dobrar de valor, usando `while`.

Exercício 19 — Acesso seguro em dicionário aninhado

Reaproveite `pib_por_ano` do Exercício 7. Escreva `pib_seguro(dados, pais, ano)` que retorna o valor se país e ano existirem, ou `None` — tratando tanto o caso de país inexistente quanto o de ano inexistente, com **um único bloco `try/except`** (dica: o erro é o mesmo tipo nos dois casos).

Exercício 20 — Projeto integrador: painel mensal

Este exercício combina praticamente tudo do capítulo. Dada a lista de dicionários abaixo (um por mês), escreva um programa que: (1) calcule o juro real de cada mês (`selic - ipca`); (2) identifique o mês com maior IPCA; (3) conte quantos meses tiveram juro real negativo; (4) imprima um resumo formatado.

```
painel = [
    {"mes": "jan", "selic": 10.5, "ipca": 0.42},
    {"mes": "fev", "selic": 10.5, "ipca": 0.83},
    {"mes": "mar", "selic": 10.75, "ipca": 0.56},
    {"mes": "abr", "selic": 10.75, "ipca": 0.44},
    {"mes": "mai", "selic": 11.0, "ipca": 1.20},
]
```

Capítulo 17 — Resumo

Conceito	Sintaxe
Print	<code>print(f"texto {var}")</code>
Variável (rótulo, não caixa)	<code>x = 10</code>
Lista (mutável)	<code>lista = [1, 2, 3]</code>
Dicionário (hash, acesso por chave)	<code>d = {"chave": valor}</code>
If/elif/else	<code>if x > 0: / elif / else:</code>
Loop for (iteráveis)	<code>for item in lista:</code>
Loop while (condição)	<code>while condicao:</code>
Função	<code>def nome(x): return x</code>
List comprehension	<code>[x*2 for x in lista if x > 0]</code>
Try/except	<code>try: ... except TipoDeErro: ...</code>
Comentário	<code># isso é comentário</code>

Próxima apostila

Quando terminar os exercícios acima, siga para a **Apostila 2 — pandas para Economia**. Lá você vai aprender a abrir tabelas, filtrar dados, agrupar, juntar bases e trabalhar com datas — 90% do que um economista faz no dia a dia. Os dicionários aninhados do Capítulo 8 vão reaparecer ali, agora como `DataFrame`.